

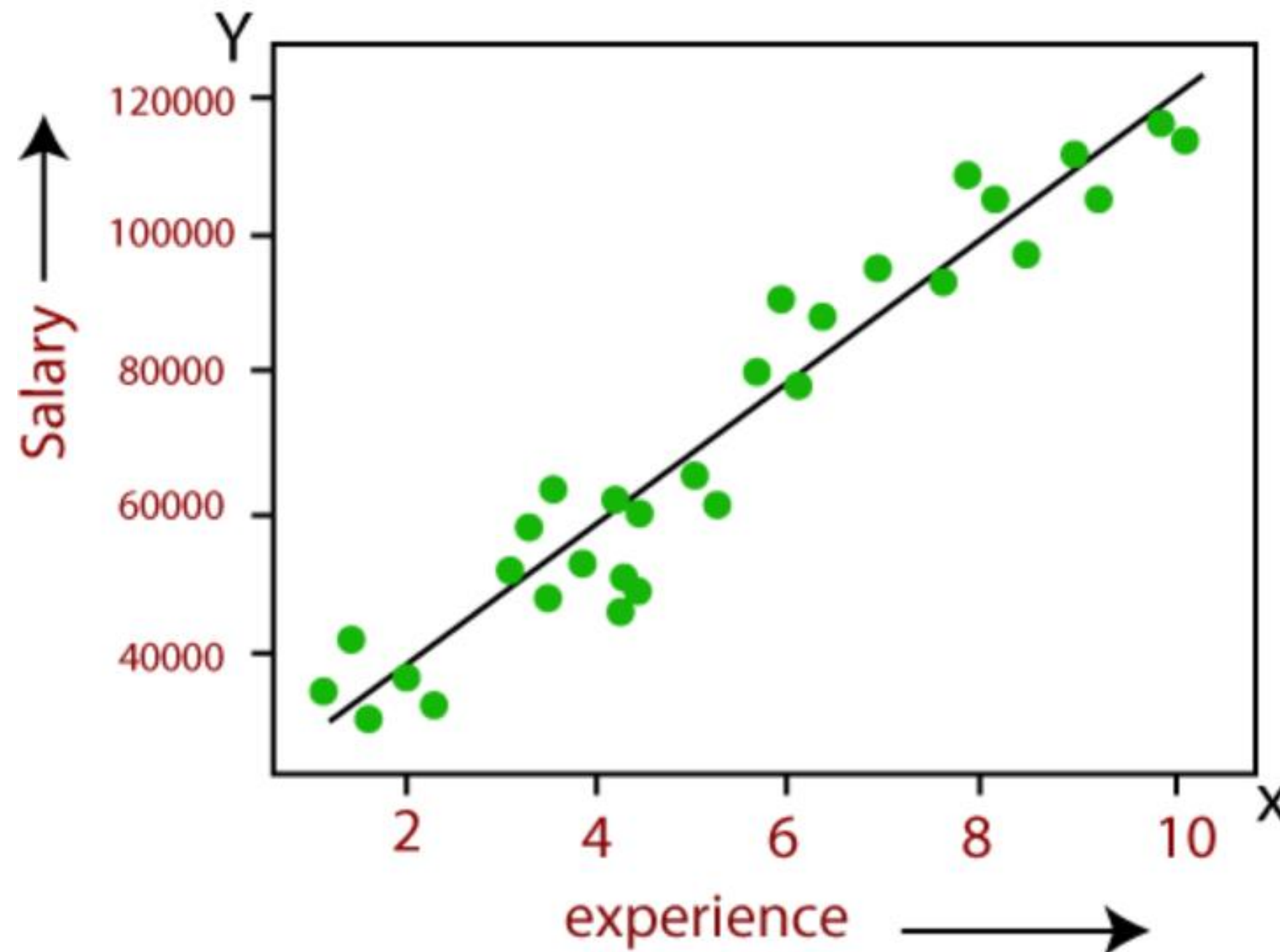
Regression Models

Regression

- Regression is a statistical technique that helps us to understand and predict the relationship between variables.
- It describes how one variable (dependent variable) changes as another variable (independent variable) changes.
 - ✓ Dependent variable: we are trying to predict or explain (Y)
 - ✓ Independent variable: that are used to predict or explain the changes in the dependent variable (X)

For example: predicting salary based on years of experience, predicting exam score based on study hours, predicting resale price based on vehicle age.

Ex.: Predicting the salary of an employee on the basis of the year of experience.



Regression algorithms

- Ordinal regression
- Poisson regression
- Fast forest quantile regression
- Linear, Polynomial, Lasso, Stepwise, Ridge regression
- Bayesian linear regression
- Neural network regression
- Decision forest regression
- Boosted decision tree regression
- KNN (K-nearest neighbors)

Regression

- Regression in machine learning consists of mathematical methods that allow data scientists to predict a continuous outcome (y) based on the value of one or more predictor variables (x).

Example:

- Sales forecasting
- Satisfaction analysis
- Price Prediction
- Employment income..etc

Types of Regression Techniques/Models

There are some regression models

- Linear Regression
- Polynomial Regression
- Ridge Regression
- Lasso Regression
- Logistic Regression

Univariate and Multivariate

- to predict a single value that is univariate
- to predict multiple values , that would be a *multivariate regression* problem.

- **Linear Regression**

A regression model that examines the relationship between one independent variable (X) and one dependent variable (Y) using a straight-line equation.

$$Y = b_0 + b_1X + \epsilon$$

- **Multi-linear Regression (Multiple Linear Regression)**

A regression model that examines the relationship between two or more independent variables and a single dependent variable.

$$Y = b_0 + b_1X_1 + b_2X_2 + \dots + b_nX_n + \epsilon$$

- **Polynomial Regression**

A regression model where the relationship between the independent variable(s) and the dependent variable is modeled as an nth degree polynomial.

$$Y = b_0 + b_1X + b_2X^2 + b_3X^3 + \dots + b_nX^n + \epsilon$$

Simple Linear Model

General Equation for Linear Regression Model is written as:

$$Y = a + bx$$

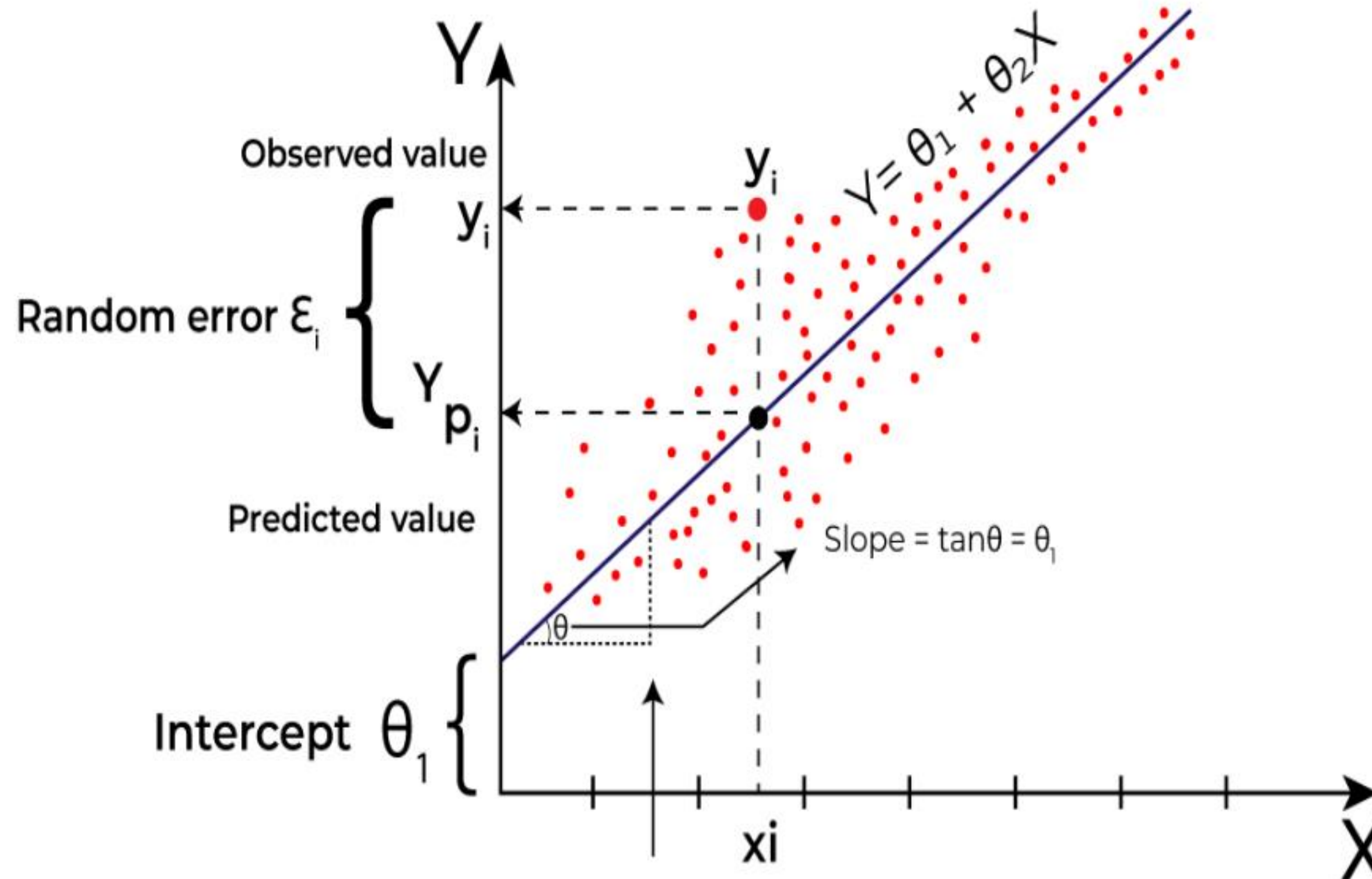
$$a = \frac{[(\Sigma y)(\Sigma x^2) - (\Sigma x)(\Sigma xy)]}{[n(\Sigma x^2) - (\Sigma x)^2]}$$

$$b = \frac{[n(\Sigma xy) - (\Sigma x)(\Sigma y)]}{[n(\Sigma x^2) - (\Sigma x)^2]}$$

where,

- y is the Dependent Variable that Lies along the Y-axis
- a is Y-Intercept
- b is the Slope of the [Regression Line](#)
- x is an Independent Variable that Lies along the X-axis

Simple Linear Model



Question 1: Find the linear regression equation for the given data:

x	y
3	8
9	6
5	4
3	2

x	y	x^2	xy
3	8	9	24
9	6	81	54
5	4	25	20
3	2	9	6
$\sum x = 20$	$\sum y = 20$	$\sum x^2 = 124$	$\sum xy = 104$

Using formula,

$$a = \frac{\sum y \sum x^2 - \sum x \sum xy}{n(\sum x^2) - (\sum x)^2}$$

$$a = \{20 (124) - 20 (104)\} / \{4 (124) - 400\}$$

$$a = 400/96 = 4.17$$

$$b = \frac{n \sum xy - (\sum x)(\sum y)}{n \sum x^2 - (\sum x)^2}$$

$$b = \{4 (104) - 20 (20)\} / \{4 (124) - 400\}$$

$$b = 16/96 = 0.166$$

So, linear regression equation is, $y=a+bx \rightarrow y = 4.17 + 0.167x$

Question 2: Find the linear regression equation for the given data:

x	y
4	6
7	5
3	8
1	3

Using formula,

$$a = \frac{\sum y \sum x^2 - \sum x \sum xy}{n(\sum x^2) - (\sum x)^2}$$

$$= (22 (75) - 15 (86)) / (4 (75) - 225)$$

$$= 360/75$$

$$= 4.8$$

$$b = \frac{n \sum xy - (\sum x)(\sum y)}{n \sum x^2 - (\sum x)^2}$$

$$= (4 (86) - 15 (22)) / (4 (75) - 225)$$

$$= 14/75$$

$$= 0.1867$$

*So, the linear regression equation is, **$y = 4.8 + 0.187x$** .*

Mathematics

$$Y = 2x + 3$$

$$x = [1, 2, 3, 4, 5]$$

$$Y = [5, 7, 9, 11, 13]$$

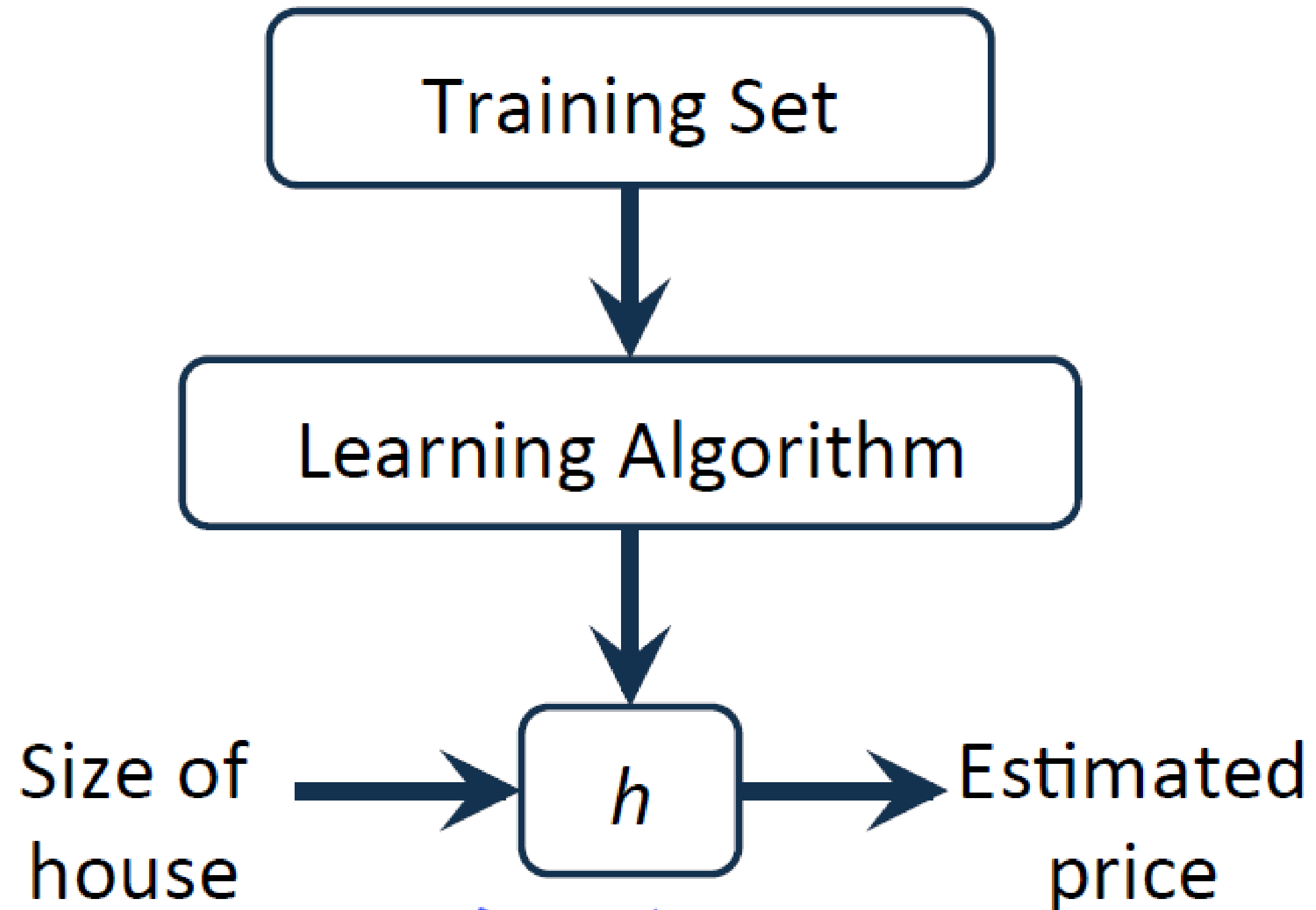
Machine Learning

$$Y = [5, 7, 9, 11, 13]$$

$$x = [1, 2, 3, 4, 5]$$

$$Y = 2x + 3$$

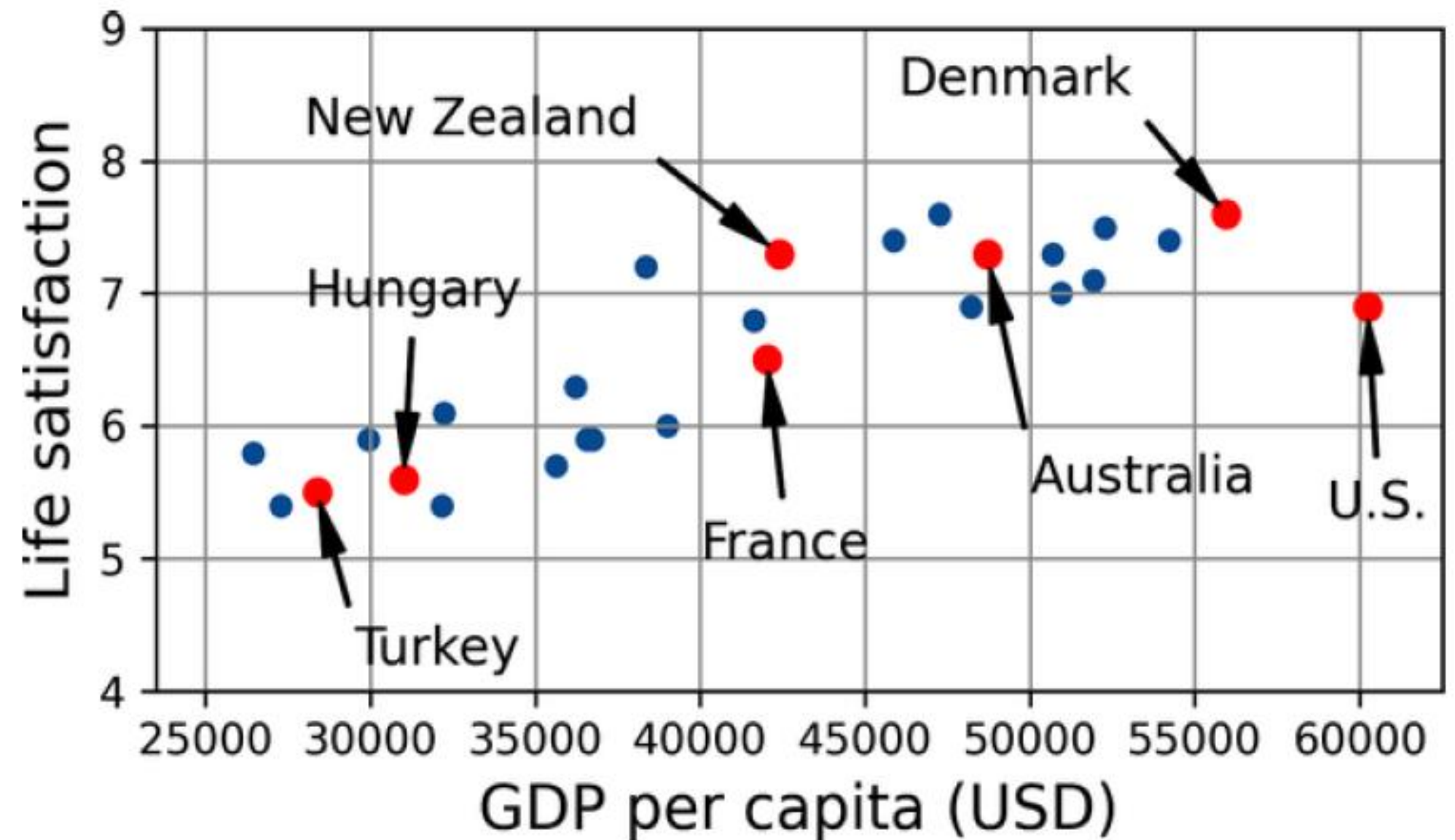
Hypothesis function in Linear Regression



Simple Linear Regression

Table 1-1. Does money make people happier?

Country	GDP per capita (USD)	Life satisfaction
Turkey	28,384	5.5
Hungary	31,008	5.6
France	42,026	6.5
United States	60,236	6.9
New Zealand	42,404	7.3
Australia	48,698	7.3
Denmark	55,938	7.6

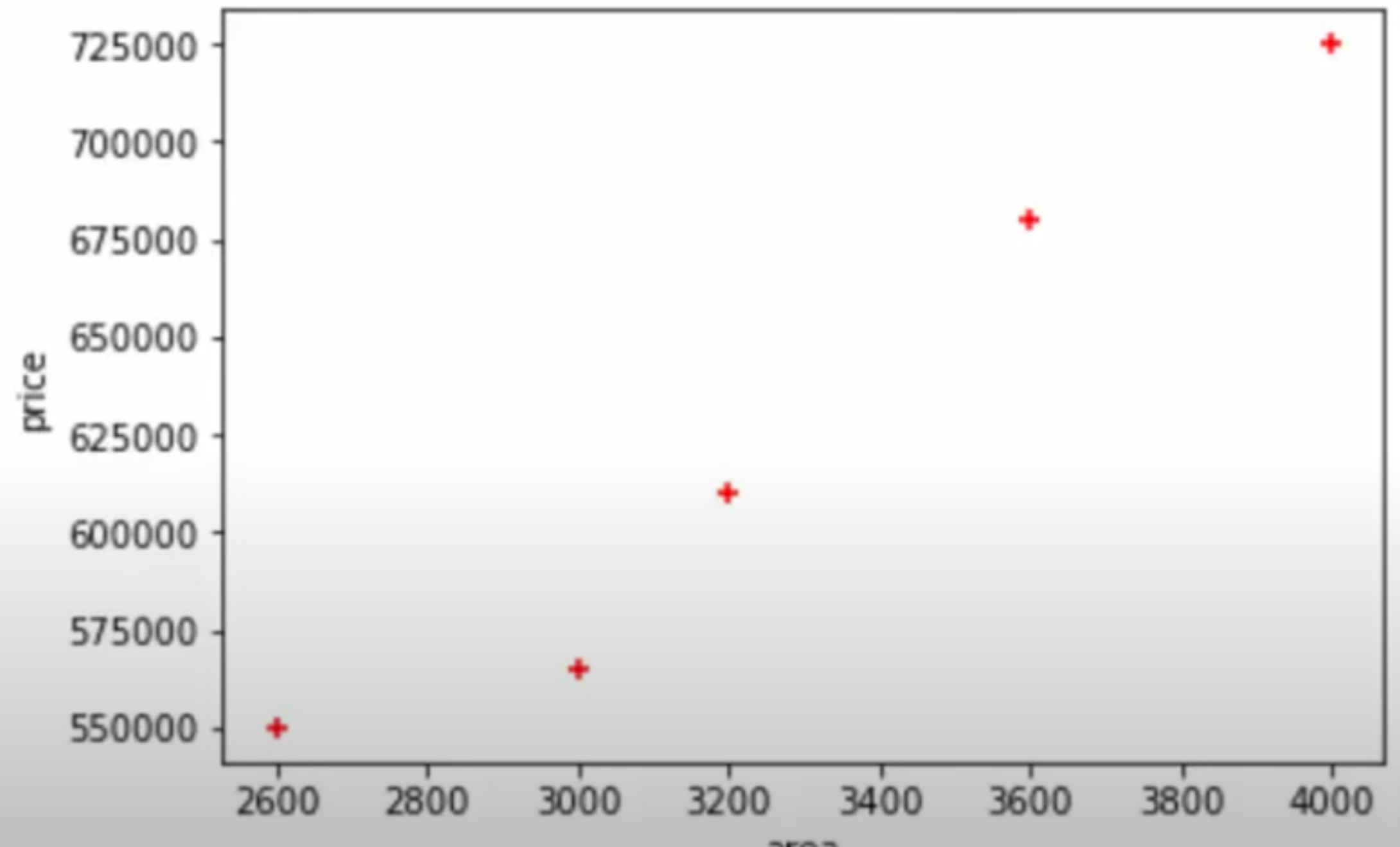


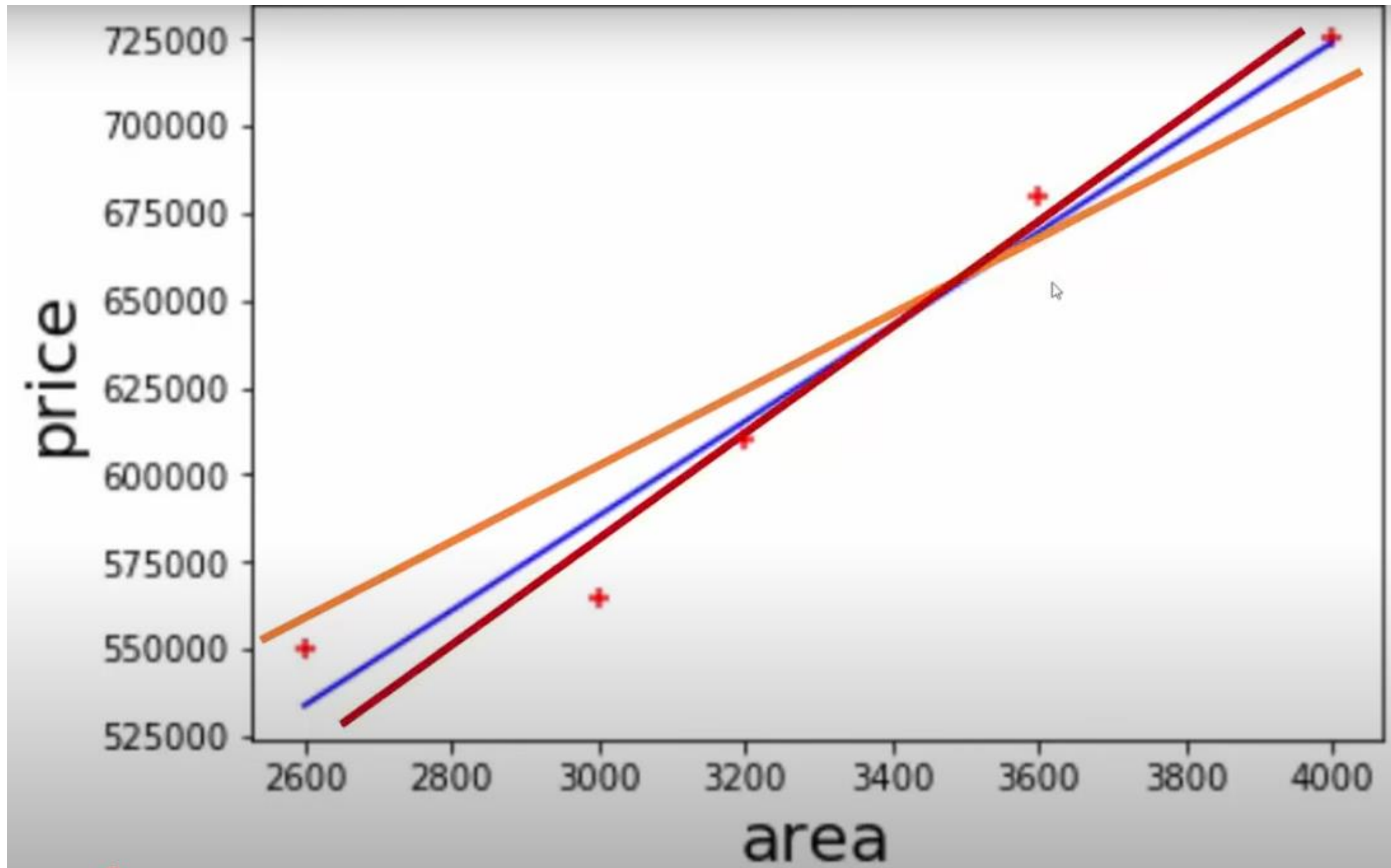
```
area = [2600, 3000, 3200, 3600, 4000]
```

```
price = [550k, 565k, 610k, 680k, 725k]
```

```
price = 135.78 * area + 180616.43
```

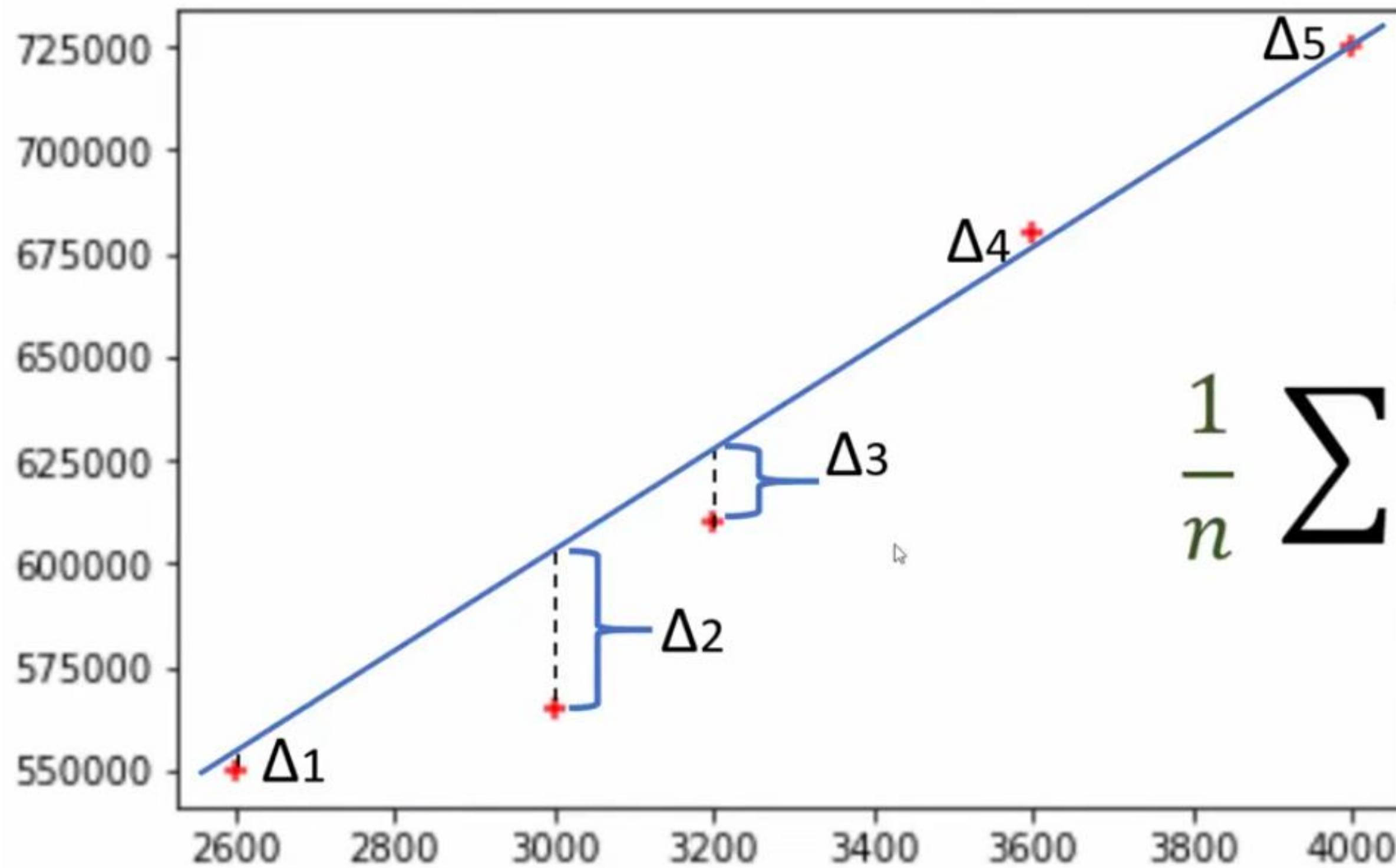
area	price
2600	550000
3000	565000
3200	610000
3600	680000
4000	725000





Best Fit Line

- The best Fit Line equation provides a straight line that represents the relationship between the dependent and independent variables.
- The primary objective while using linear regression is to locate the best-fit line, which implies that the error between the predicted and actual values should be kept to a minimum



$$\frac{1}{n} \sum_{i=1}^n (\Delta i)^2$$

Mean Squared Error

$$mse = \frac{1}{n} \sum_{i=1}^n (y_i - (mx_i + b))^2$$

Cost Function

Cost function

Cost function is the error or difference between prediction value and actual values.

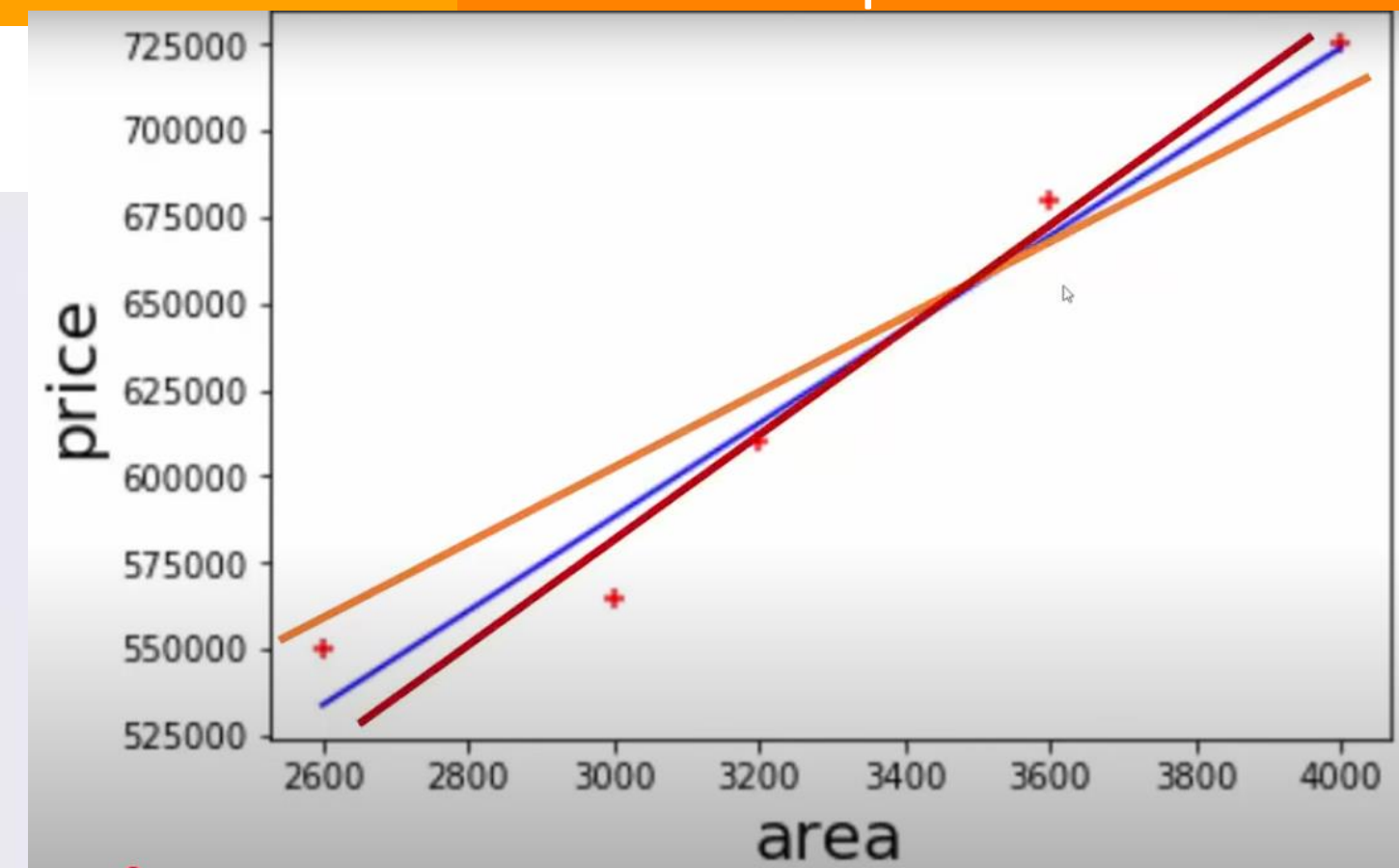
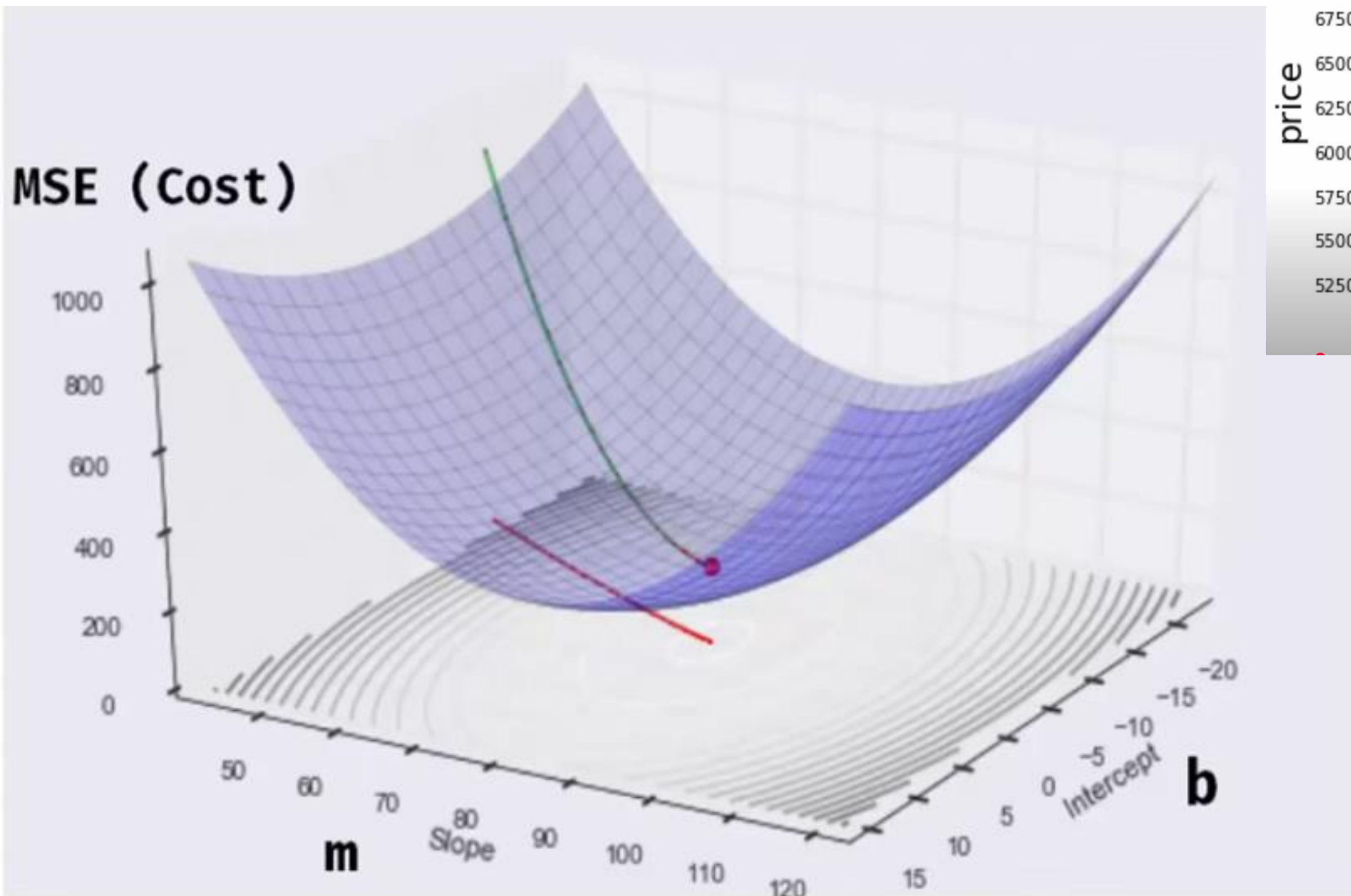
$$\text{Cost function}(J) = \frac{1}{n} \sum_n^i (\hat{y}_i - y_i)^2$$

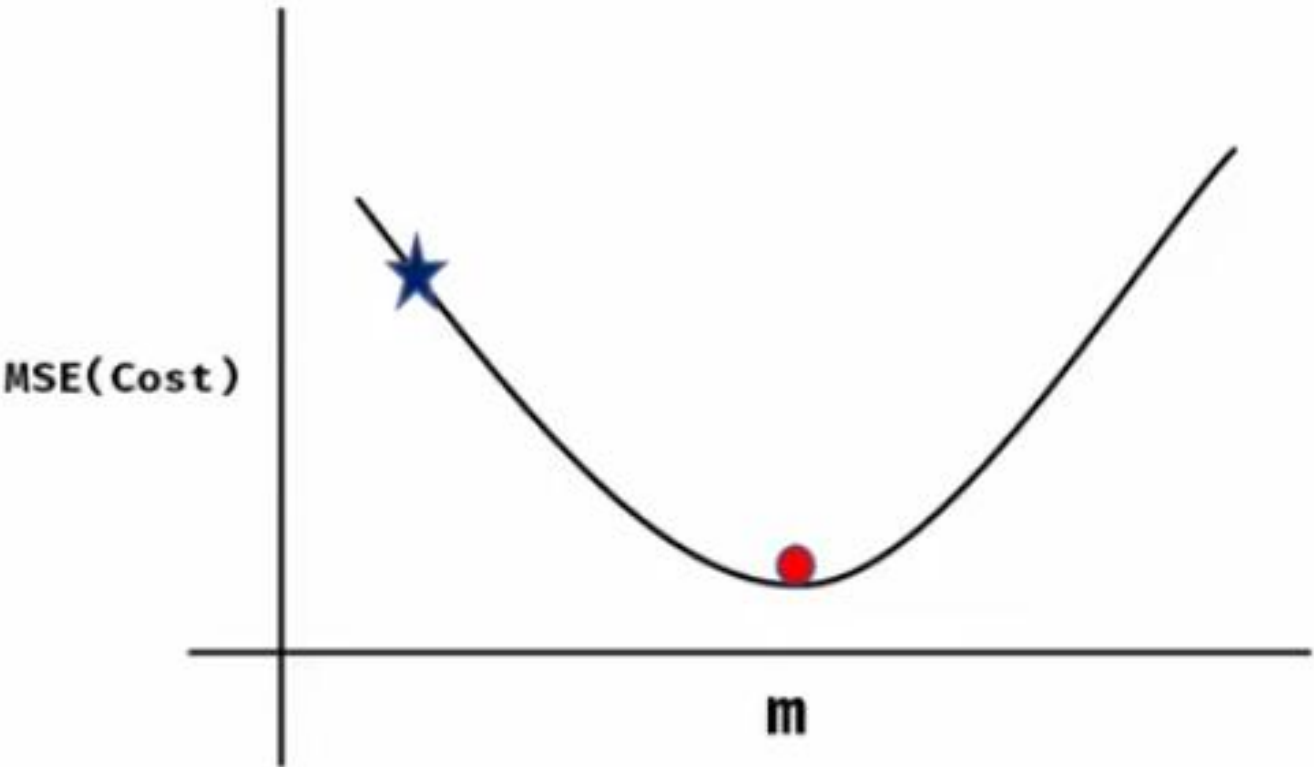
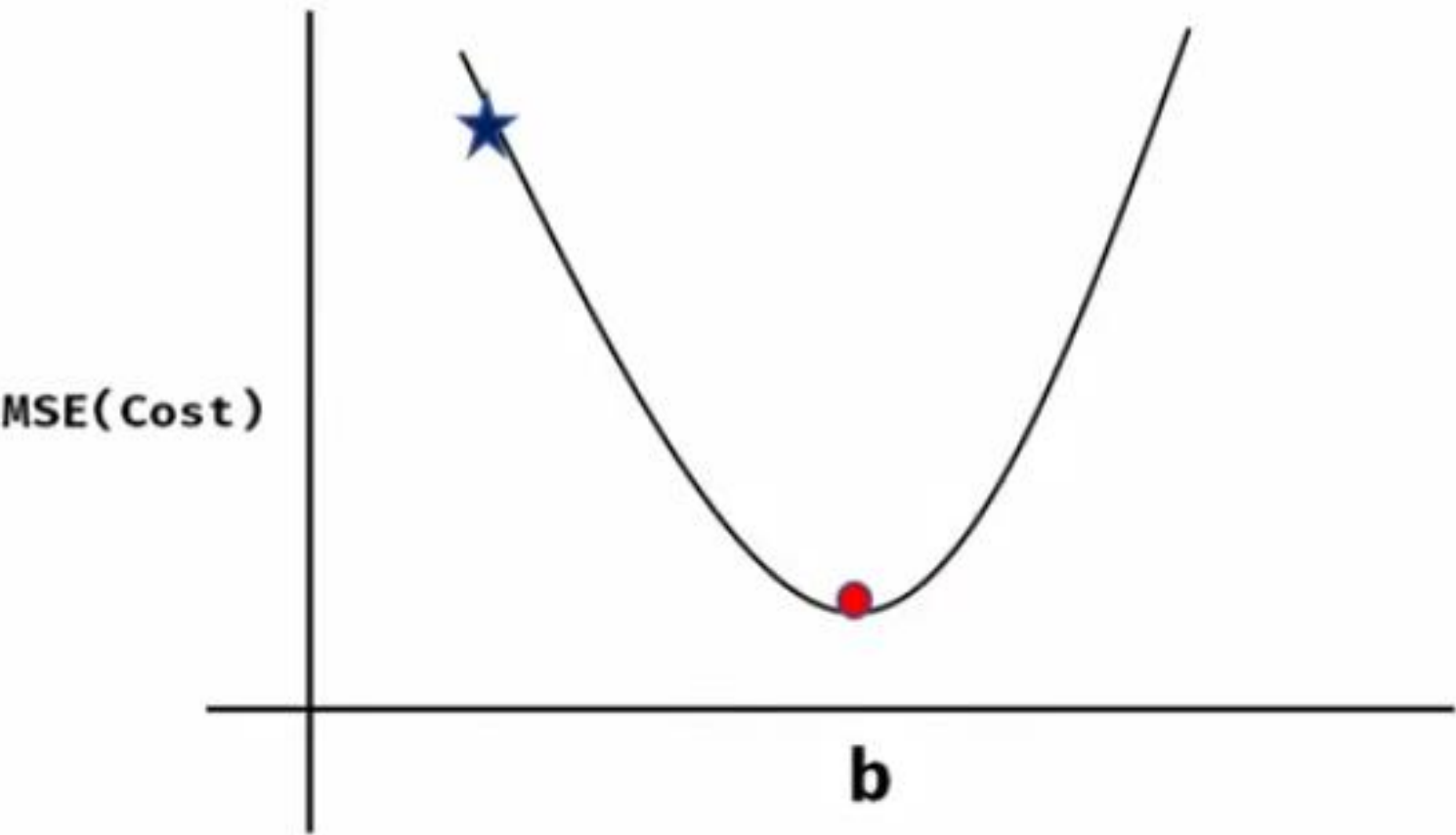
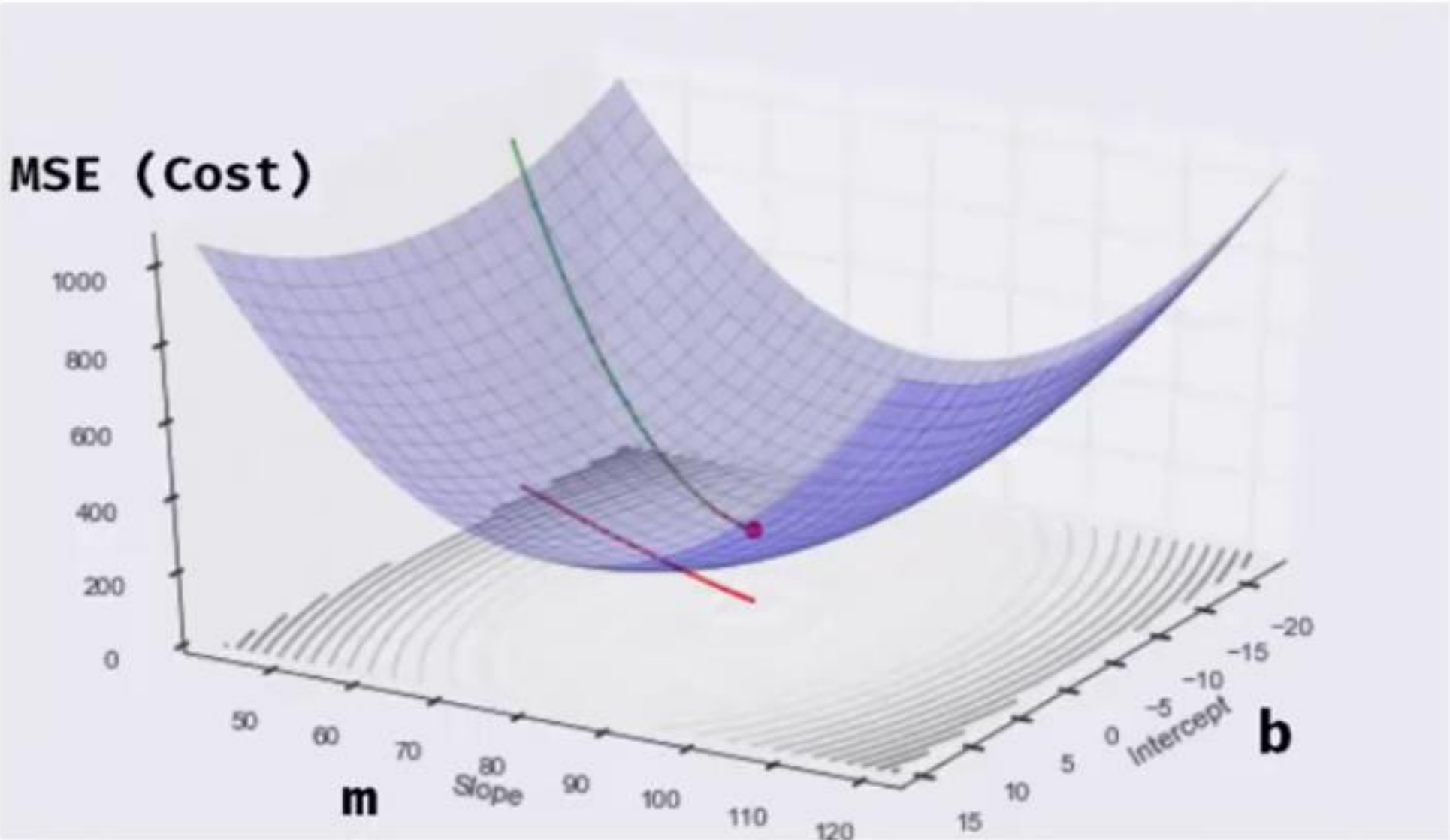
Gradient descent is an algorithm that finds best fit line for given training data set

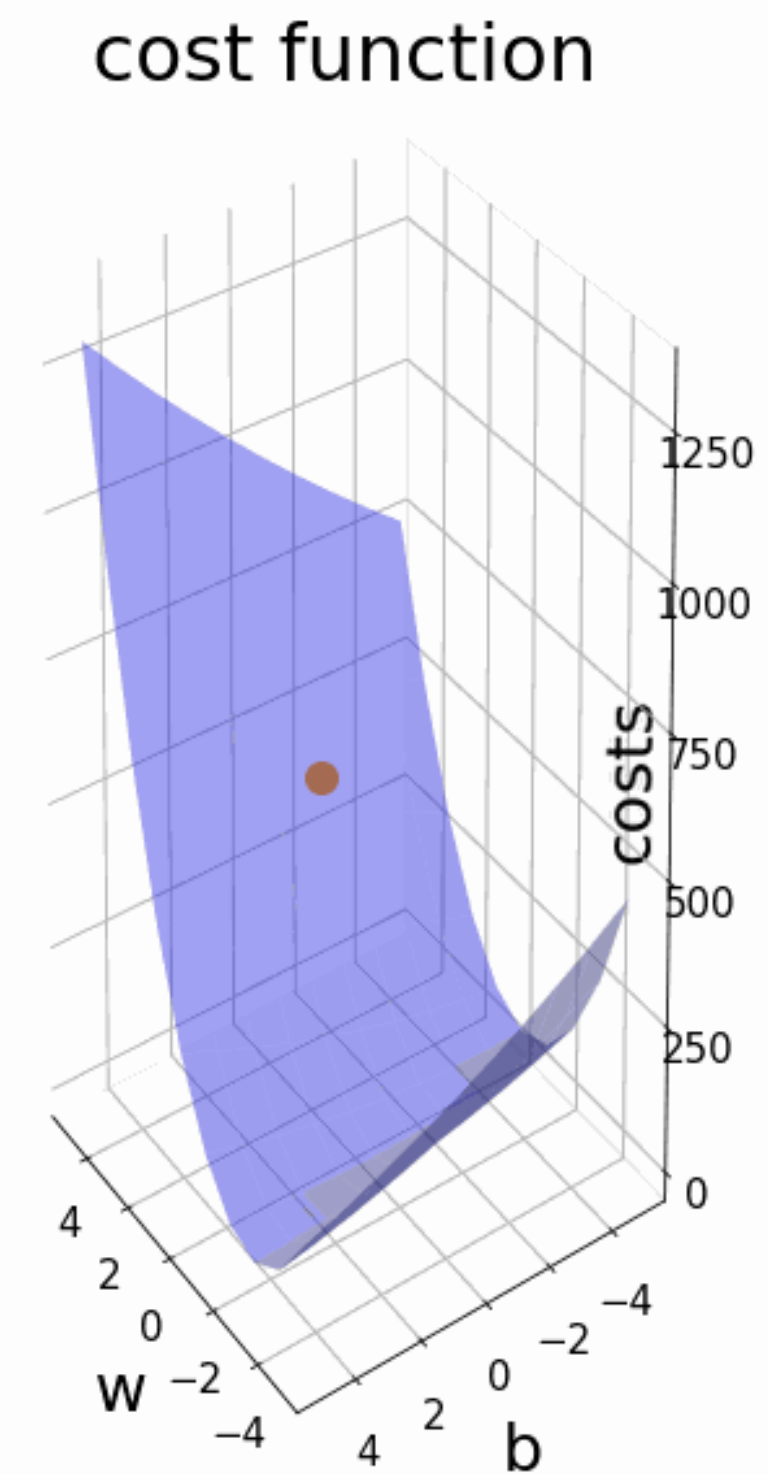
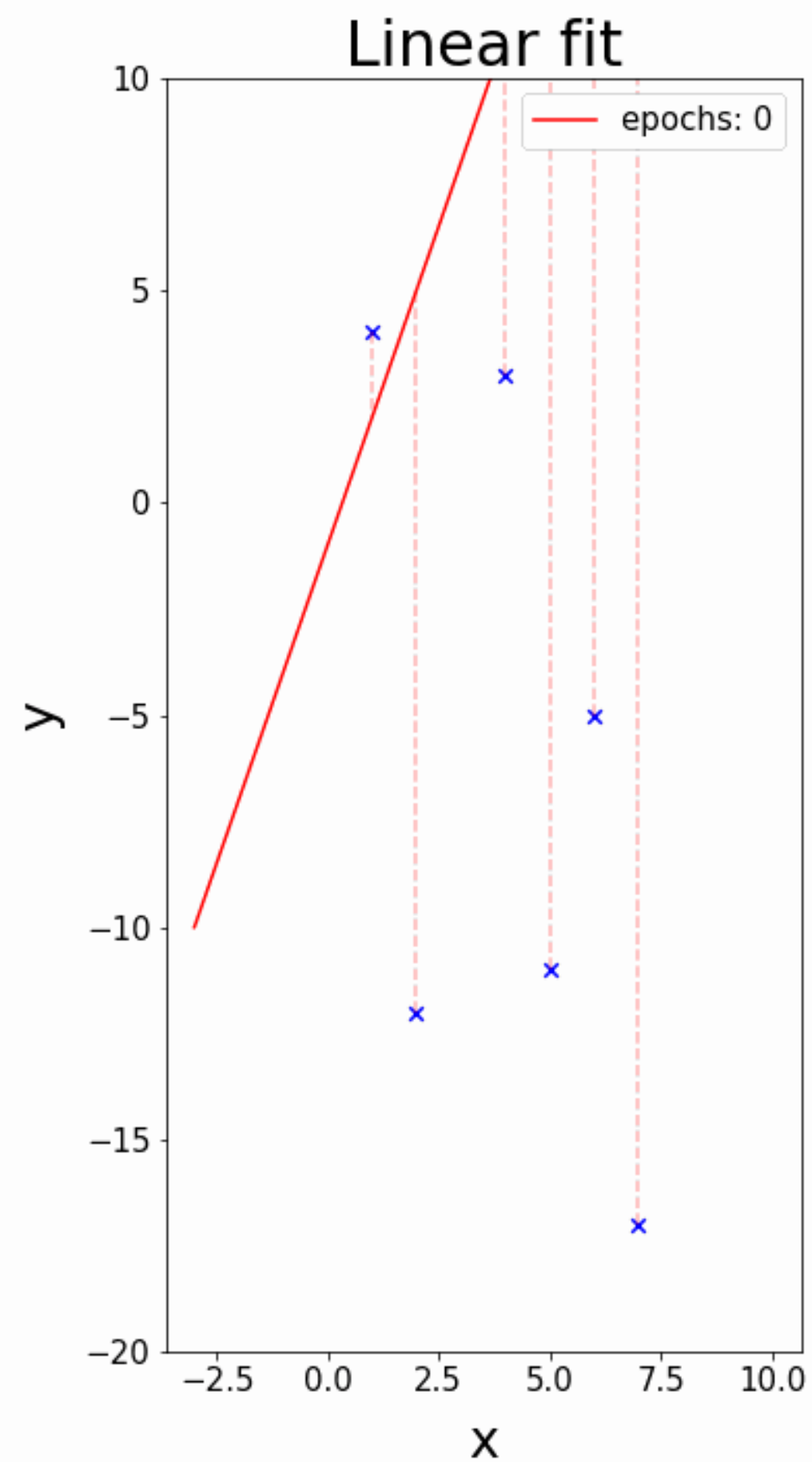
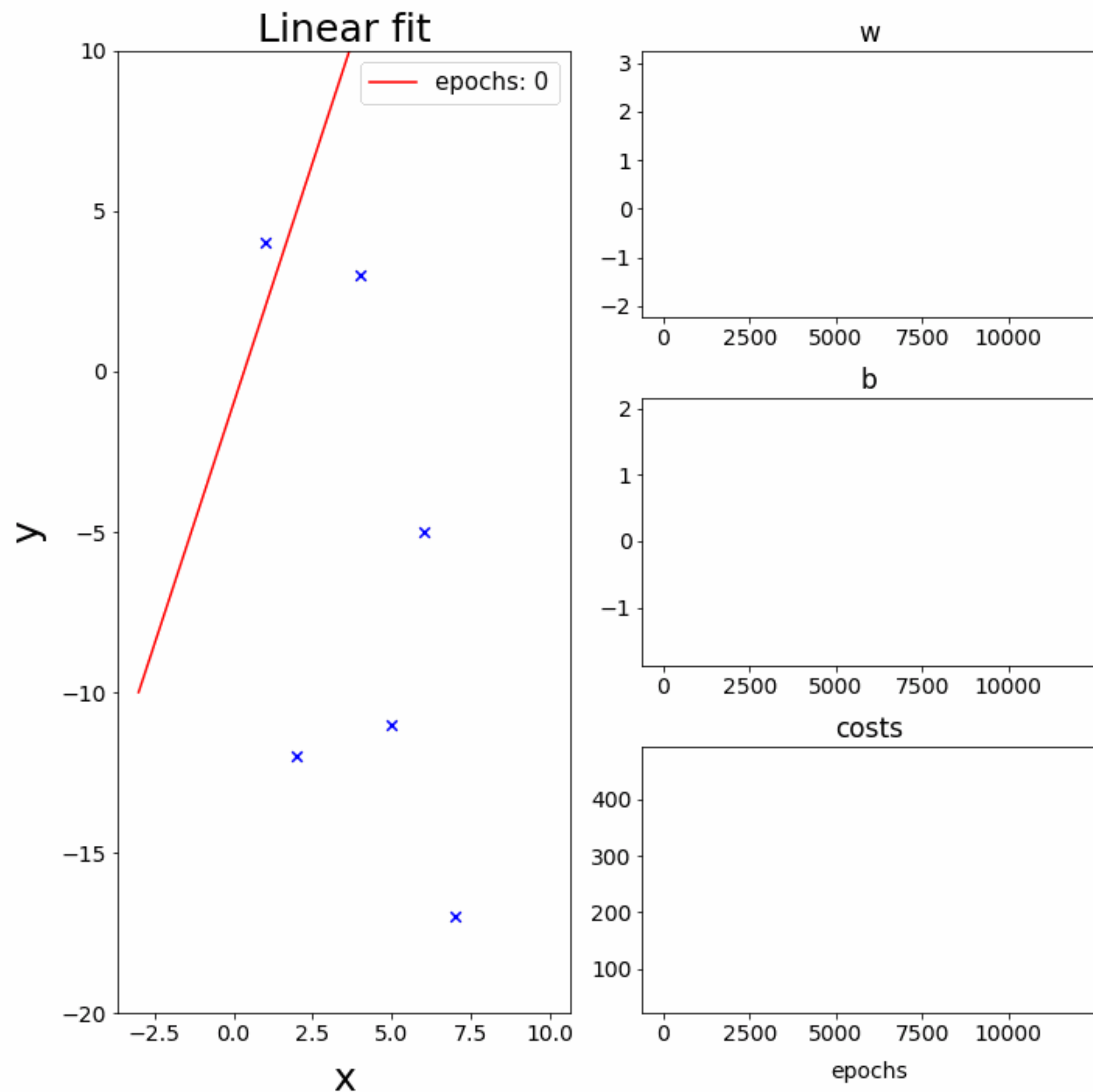
Gradient Descent

Gradient descent is a generic optimization algorithm capable of finding optimal solutions to a wide range of problems.

***Idea :** Start by filling θ with random values (this is called random initialization). Then improve it gradually, taking one baby step at a time, each step attempting to decrease the cost function (e.g., the MSE), until the algorithm converges to a minimum .*







Step Size and Learning Rate

- If the learning rate is too small, then the algorithm will have to go through many iterations to converge, which will take a long time
- If the learning rate is too high, you might jump across the valley and end up on the other side, possibly even higher up than you were before.

Step Size and Learning Rate

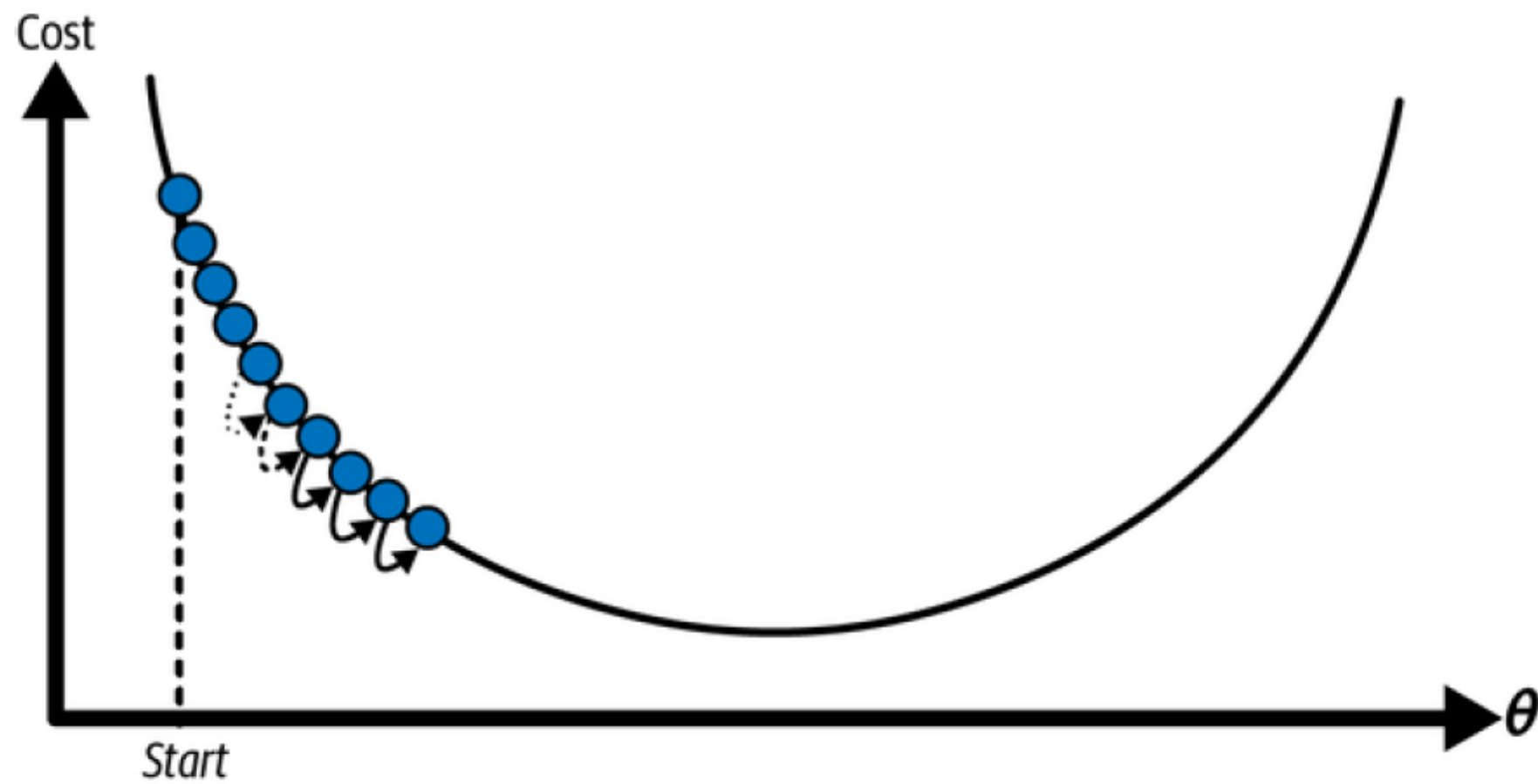


Figure 4-4. Learning rate too small

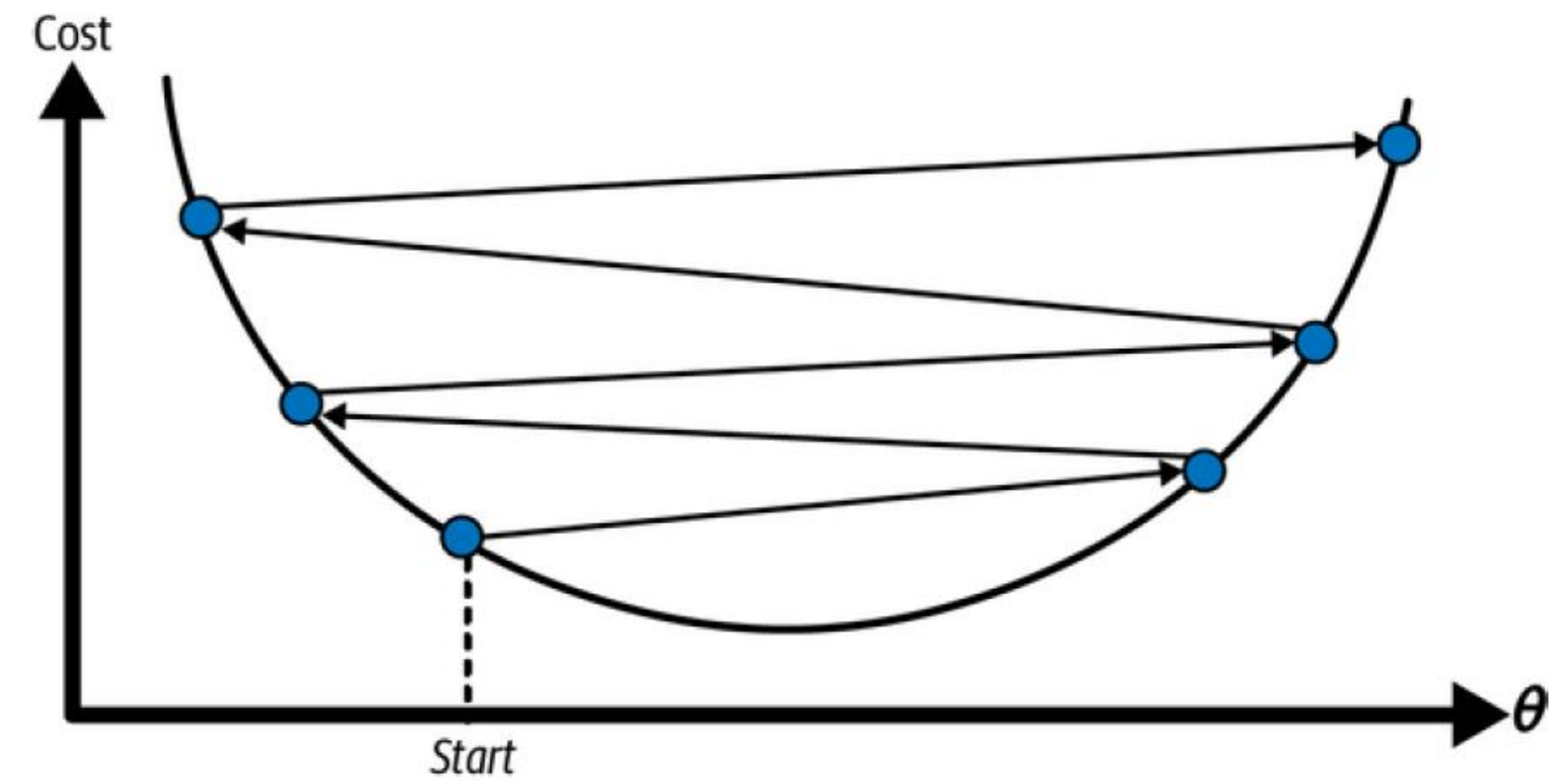
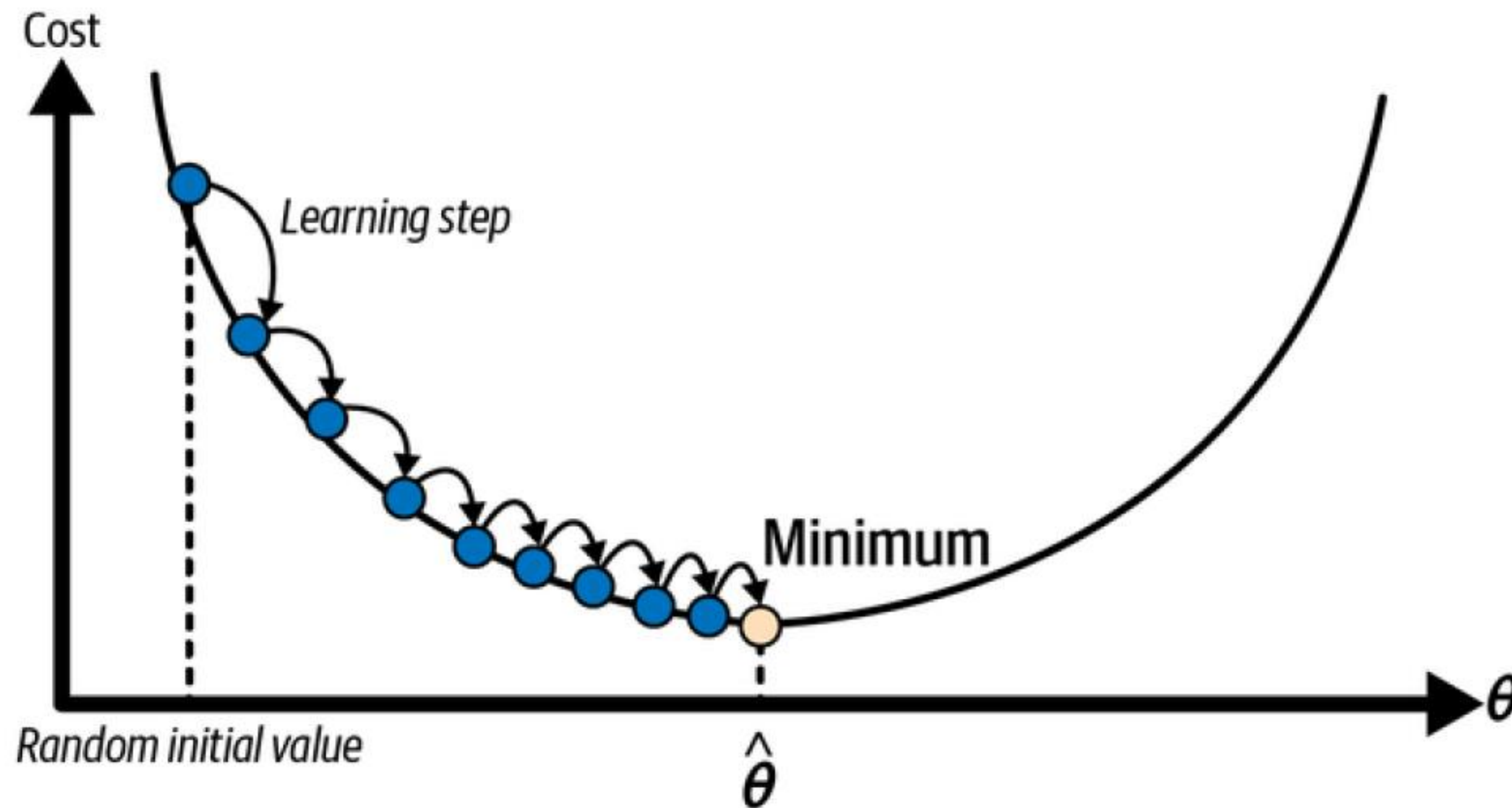
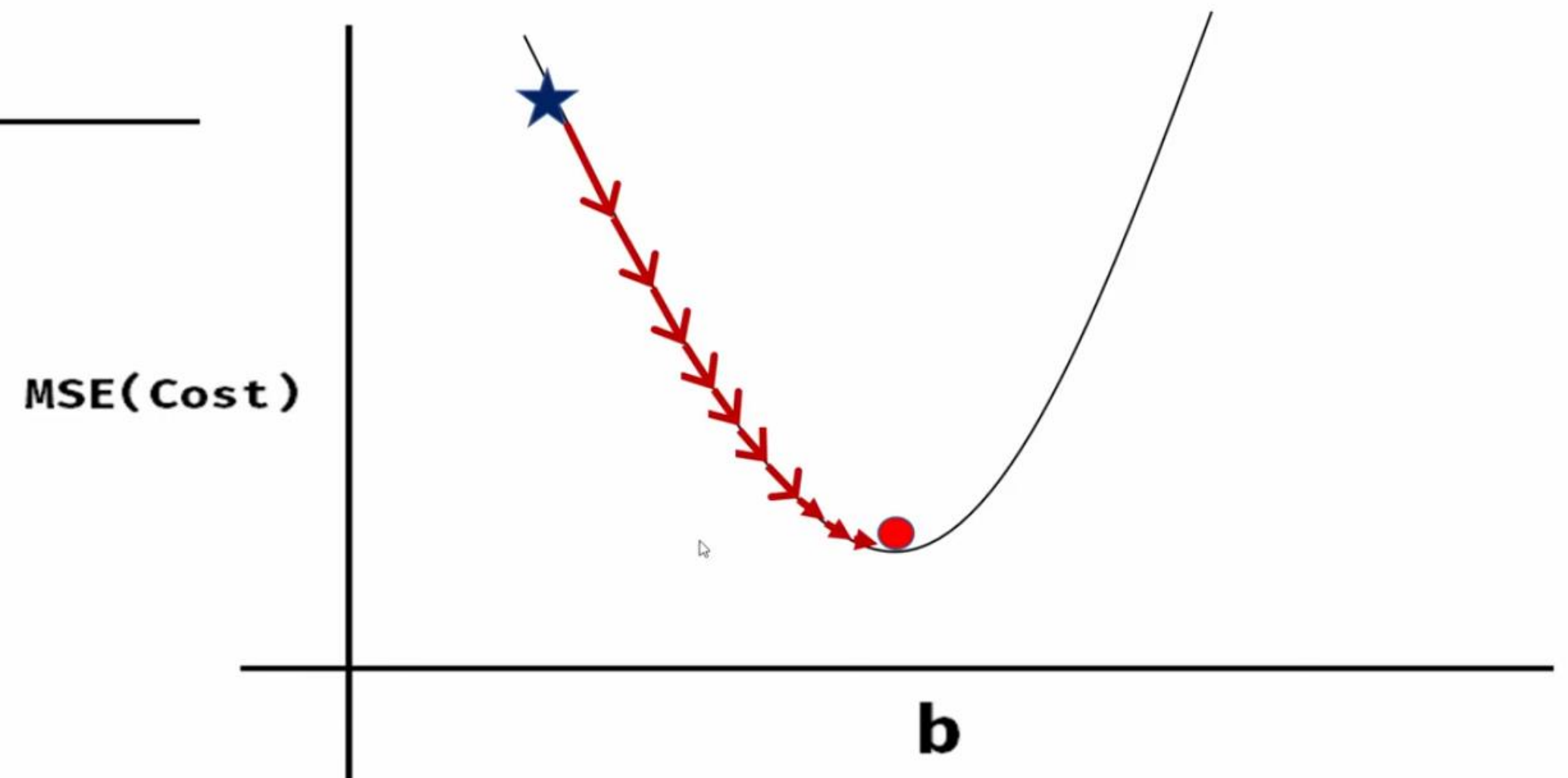
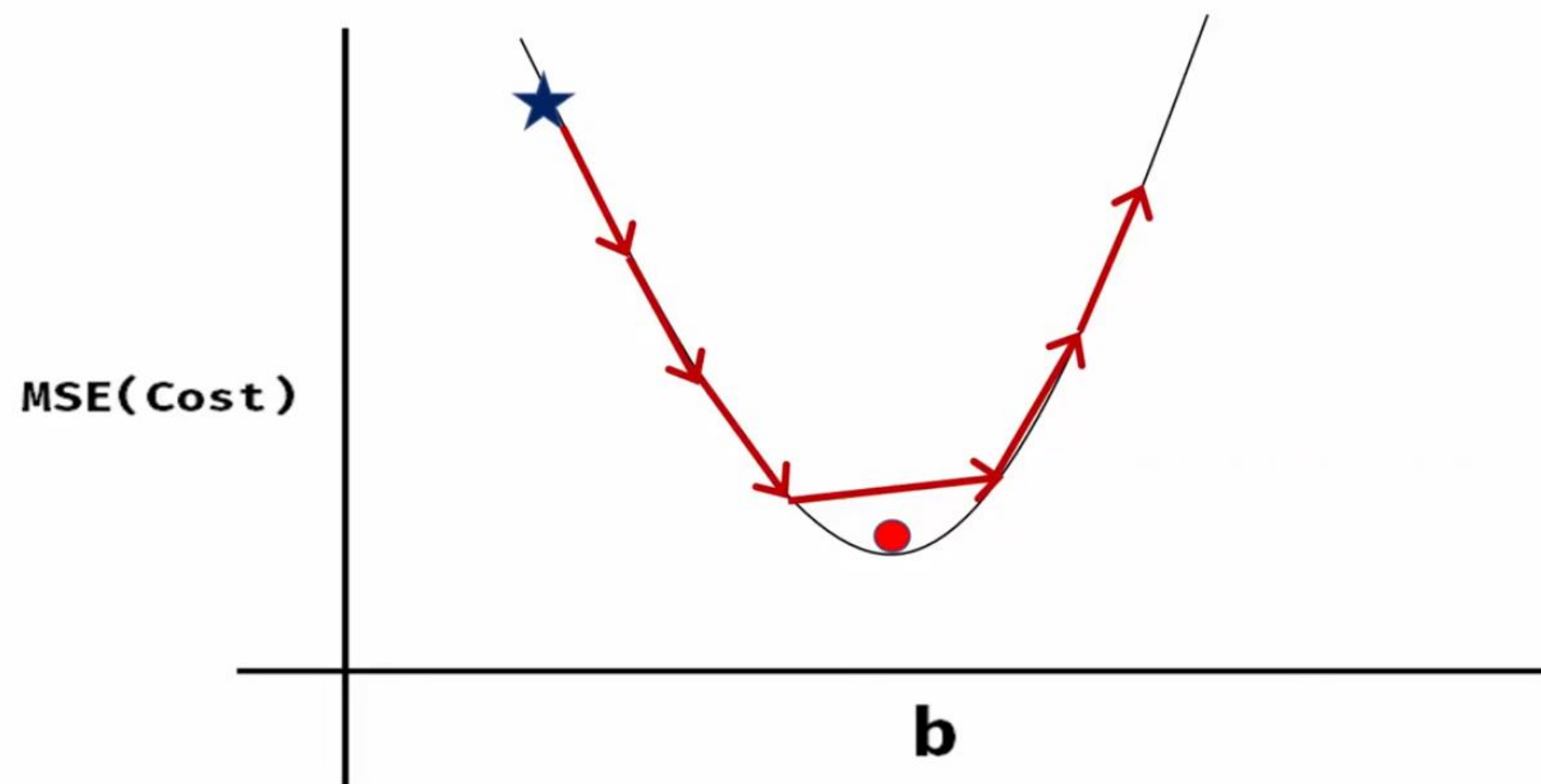


Figure 4-5. Learning rate too high

Gradient Descent





Local and Global Minima

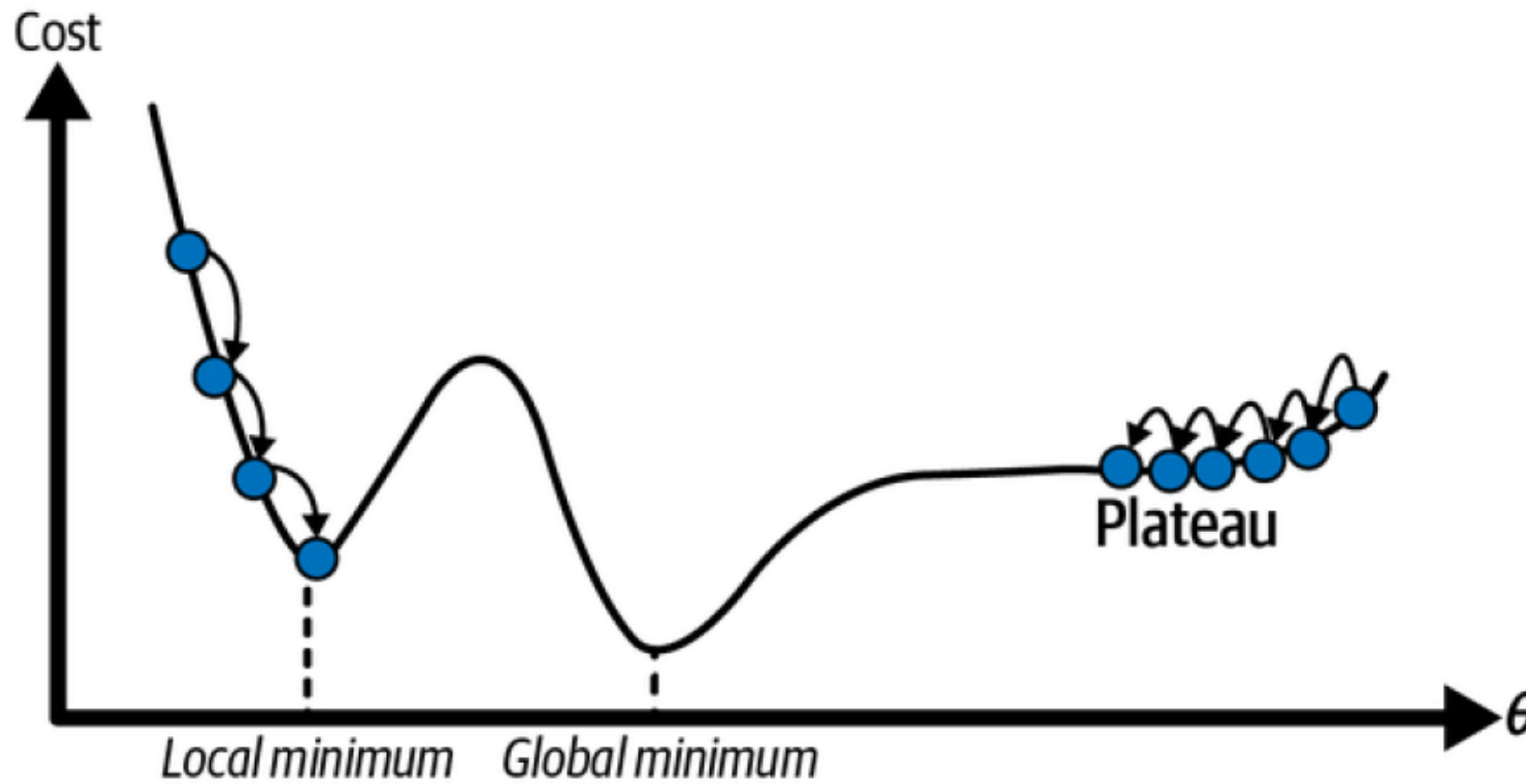
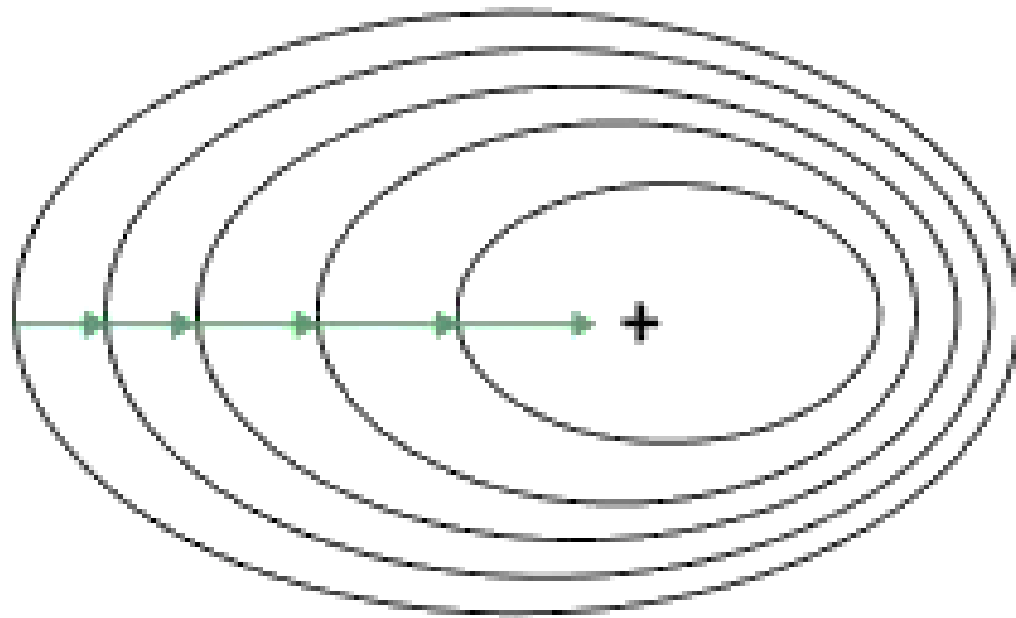


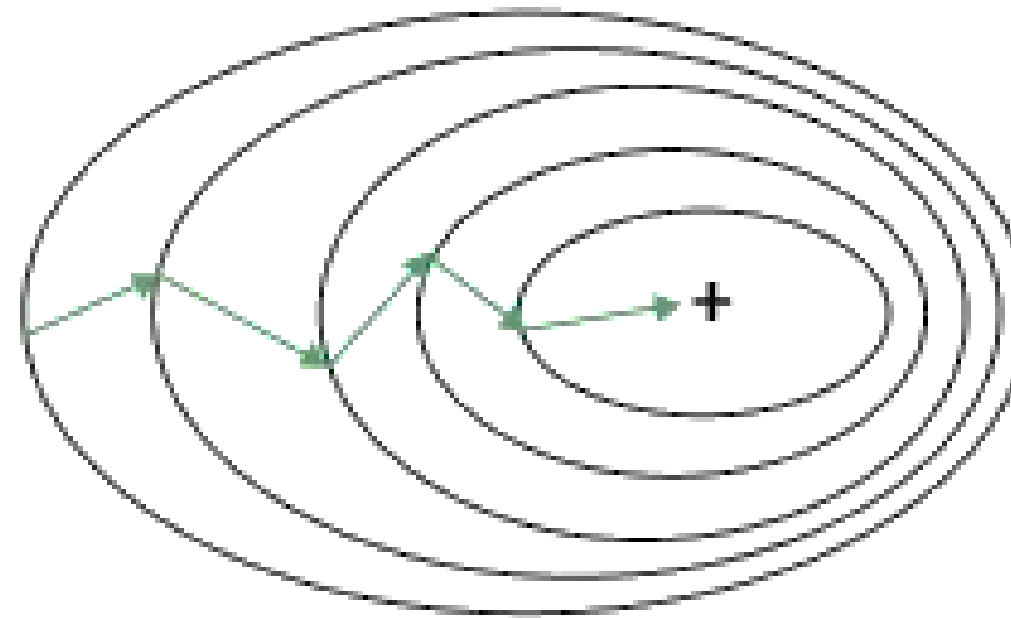
Figure 4-6. Gradient descent pitfalls

Types of Gradient Descent

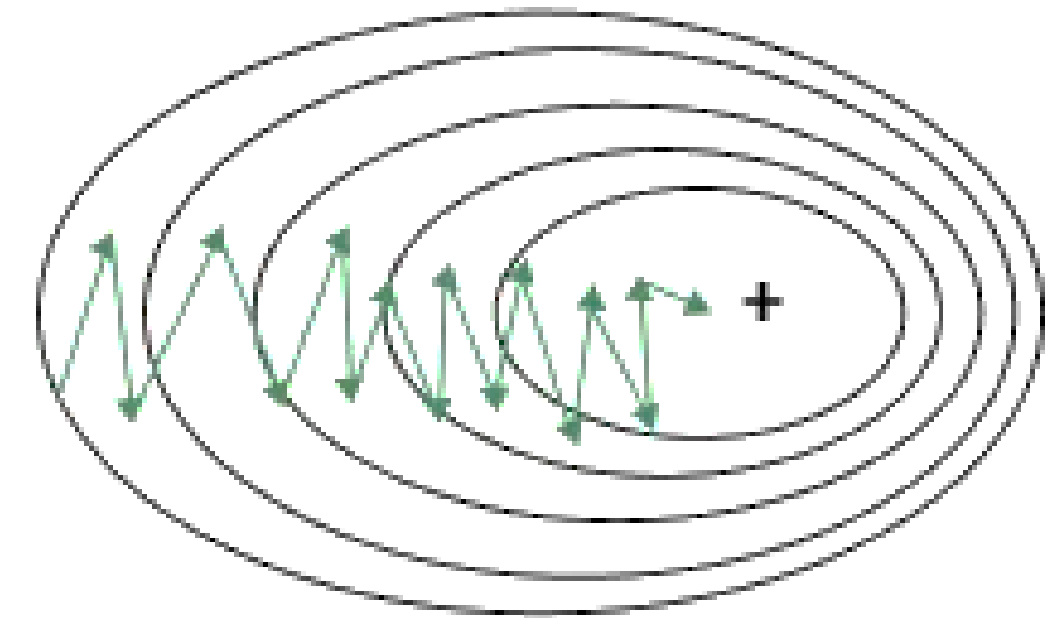
- Batch Gradient Descent
- Stochastic Gradient Descent
- Mini-Batch Gradient Descent



Batch Gradient Descent



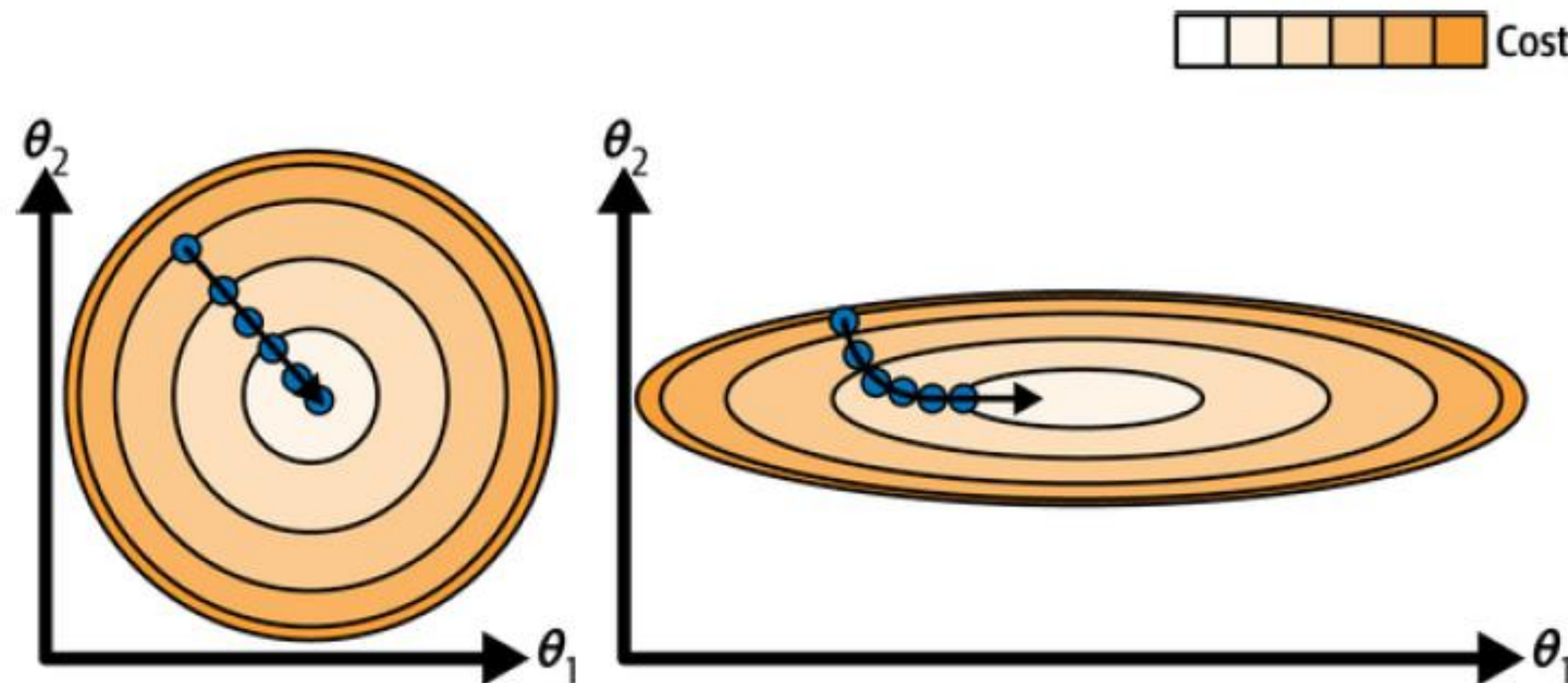
Mini-Batch
Gradient Descent



Stochastic
Gradient Descent

Batch Gradient Descent

In Batch Gradient Descent, all the training data is taken into consideration to take a single step.



Advantages of Batch Gradient Descent

- Since the gradient is averaged over all training examples the updates are less noisy and more stable.
- It considers the entire dataset for each update providing a global perspective of the loss landscape.

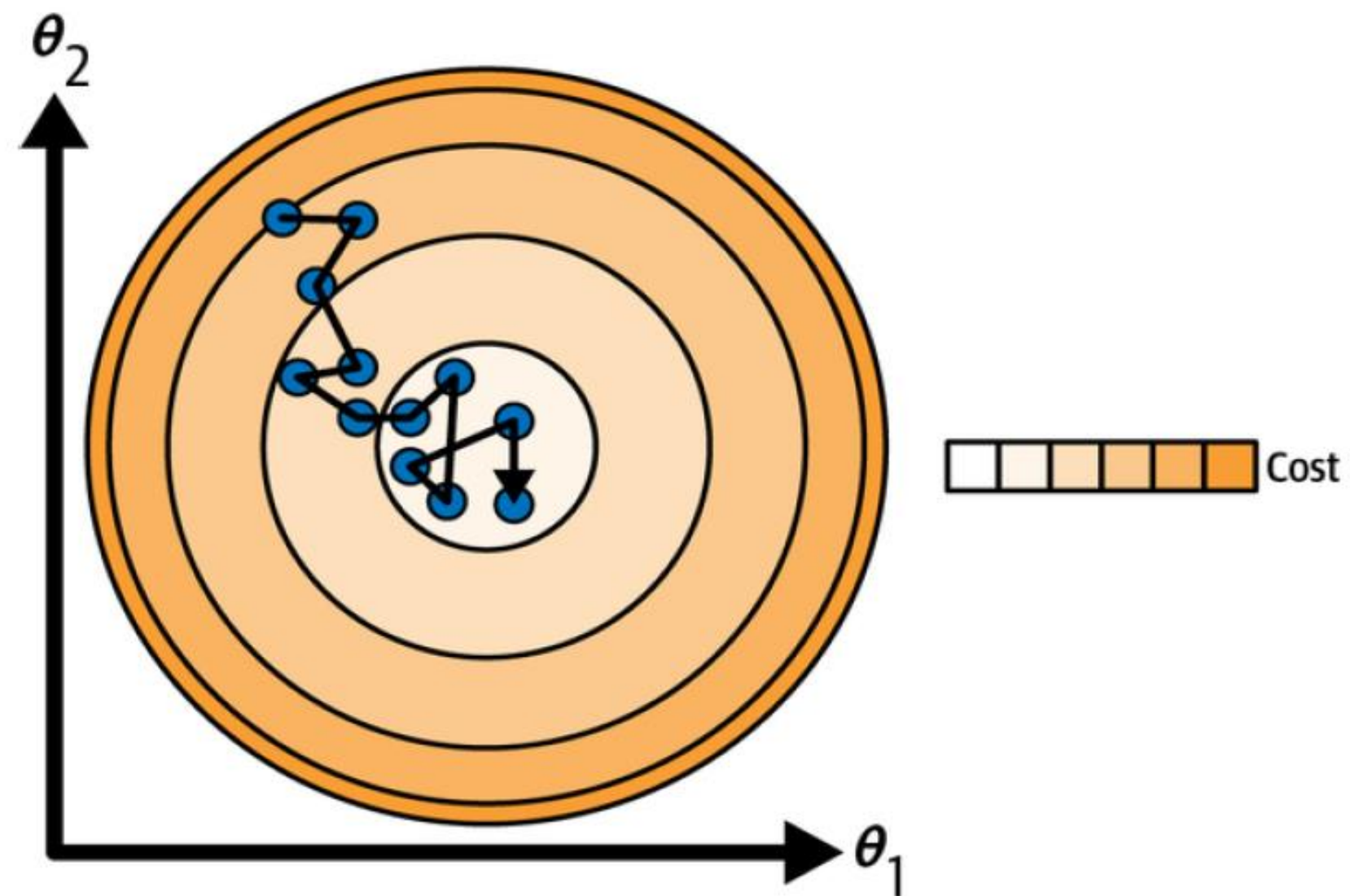
Disadvantages of Batch Gradient Descent

- **Computationally Expensive:** Processing the entire dataset in each iteration can be slow and resource-intensive especially for large datasets.
- **Memory Intensive:** This requires storing and processing the entire dataset in memory which can be impractical for very large datasets.

Stochastic Gradient Descent

In Stochastic Gradient Descent (SGD), we consider just one example at a time to take a single step.

The algorithm much faster because it has very little data to manipulate at every iteration.



Advantages of Stochastic Gradient Descent

- Frequent updates can lead to faster convergence, especially in large datasets.
- Since it processes one training example at a time, it requires less memory compared to batch gradient descent.

Disadvantages of Stochastic Gradient Descent

- Noisy Updates: Updates can be noisy, leading to a more erratic convergence path.
- Potential for Overshooting: The frequent updates can cause the algorithm to overshoot the minimum, especially with a high learning rate.
- Hyperparameter Sensitivity: Requires careful tuning of the learning rate to ensure stable and efficient convergence.

Mini Batch Gradient Descent

Mini-batch GD computes the gradients on small random sets of instances called minibatches. The main advantage of mini-batch GD over stochastic GD is that you can get a performance boost from hardware optimization of matrix operations, especially when using GPUs.

Advantages of Mini-Batch Gradient Descent

- **Faster Convergence:** By using mini-batches, it achieves a balance between the noisy updates of SGD and the stable updates of Batch Gradient Descent, often leading to faster convergence.
- **Reduced Memory Usage:** Requires less memory than Batch Gradient Descent as it only needs to store a mini-batch at a time.
- **Allows for efficient use of hardware optimizations and parallel processing,** making it suitable for large datasets.

Disadvantages of Mini-Batch Gradient Descent

- **Requires careful tuning of the mini-batch size and learning rate to ensure optimal performance.**
- **Less Stable than Batch GD:** While more stable than SGD, it can still be less stable than Batch Gradient Descent, especially if the mini-batch size is too small.

Polynomial Regression

- Add powers of each feature as new features, then train a linear model on this extended set of features. This technique is called *polynomial regression*.

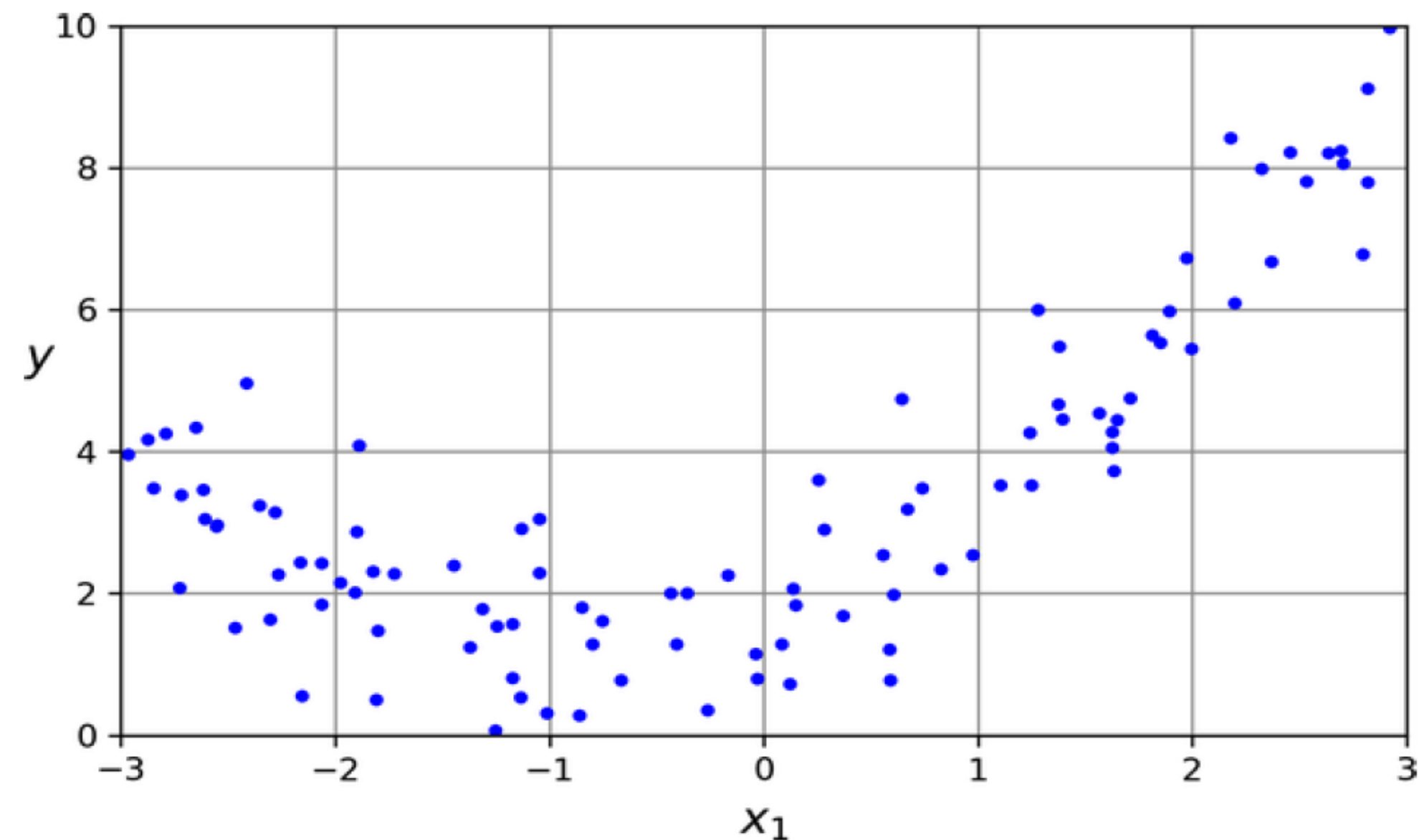


Figure 4-12. Generated nonlinear and noisy dataset

Overfitting and Underfitting

This high-degree polynomial regression model is severely overfitting the training data, while the linear model is underfitting it.

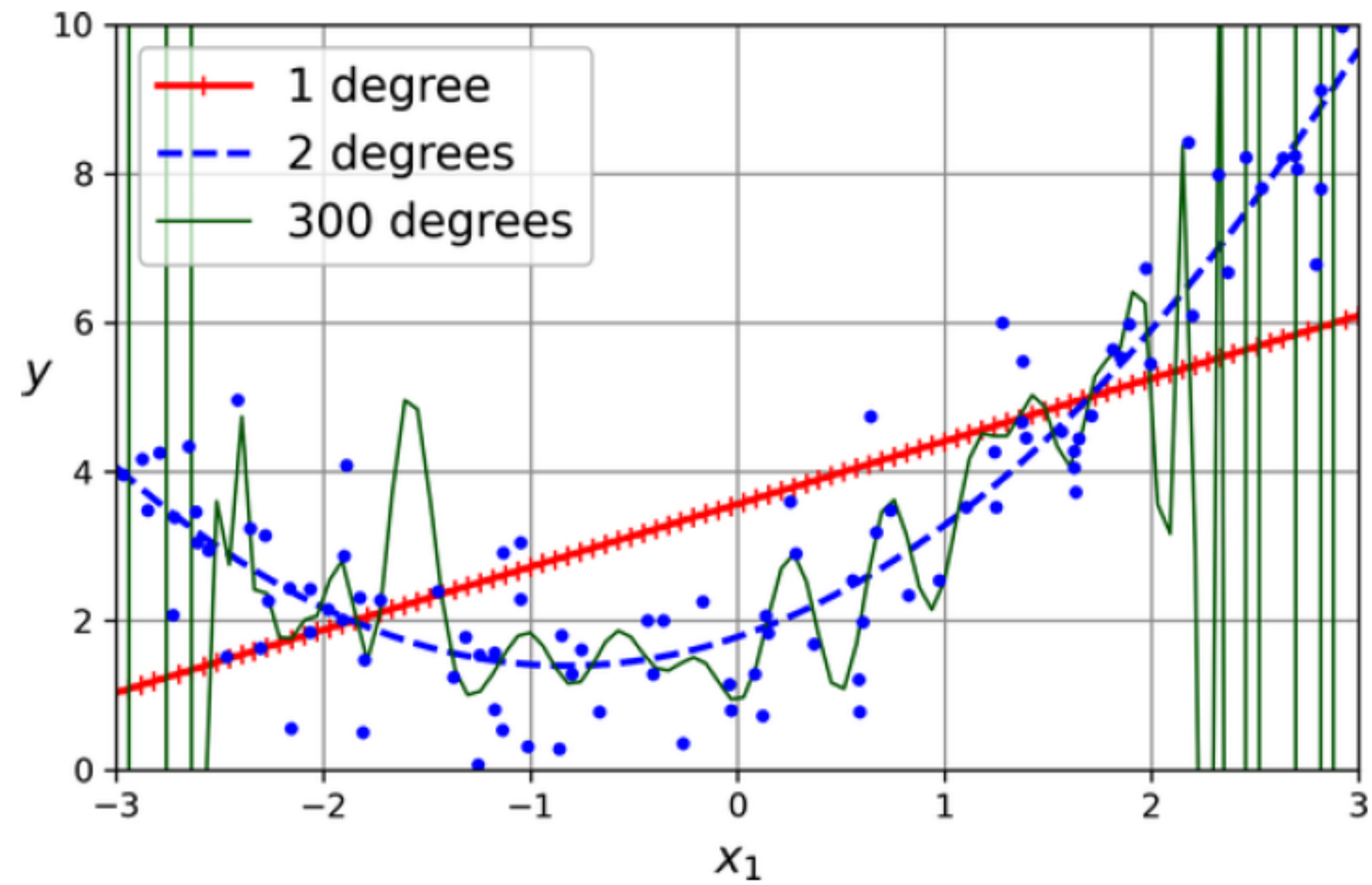


Figure 4-14. High-degree polynomial regression

Thank You!

